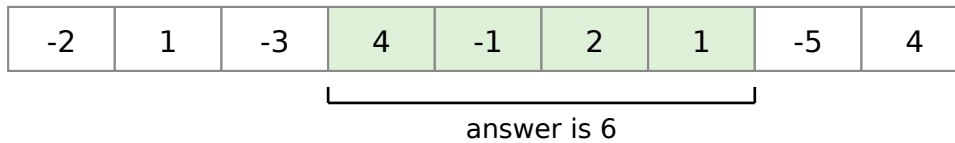


Maximum Continuous Sum

Problem: Find the max continuous sum.

Example list: [-2, 1, -3, 4, -1, 2, 1, -5, 4]



1st solution: Try every continuous slice and check the largest one. This runs in $O(n^2)$ time, but how can we improve this?

[-2, 1, -3, 4, -1, 2, 1, -5, 4]

Using 2 pointers, i and j , set them both to the first value. Since the total value is negative between the 2 pointers, reset them to the next positive value.

Then, since the current sum is positive, move the j pointer forward. But now the sum is negative again, so move both pointers to the next positive value.

Proceed, tracking every peak value between resets until you finish the list. One of these will be the highest value.

In pseudocode:

```
max_sum = -inf
cur_sum = 0
for i = 0 -> len(A):
    cur_sum += A[i]
    max_sum = max(max_sum, cur_sum)
    cur_sum = max(cur_sum, 0)
return max_sum
```

This code runs in $O(n)$ time, but now we need to prove it always outputs the correct answer.

cur_sum, at any time, stores the max sum that ends with i or, if it is negative, stores 0.

max_sum simply stores the highest that **cur_sum** has ever been, that is, from elements $A[0] \dots A[i]$.

Base case is $P(0)$:

```
cur_sum = max[0, A[0]]
max_sum = A[0]
```


Main observation: Only the majority element can be partitioned into pairs and still have an element left over.

Claim: If the votes are split into pairs of different candidates plus one leftover vote, the leftover vote must be for the majority candidate.

Proof (by contradiction): Let m be the majority candidate. Suppose the leftover vote is for some other candidate $x \neq m$.

Each pair holds two different candidates, so it contains at most one vote for m . It also contains at least one vote for someone other than m . Since the leftover vote is for x , not m , every vote for m is in a pair. So:

- votes for $m \leq$ number of pairs
- votes for non- $m \geq$ number of pairs + 1 (one per pair, plus the leftover x)

That means m has strictly fewer votes than everyone else combined, which contradicts m being the majority. So the leftover must be m . ■

Inductive Proof

Inductive claim: After k votes, we partition into paired and unpaired votes.

Case	Pairs	Candidates
Base case: $P(0)$	$\emptyset$	votes[0]
$P(i)$	(pairs)	count candidates
(i+1)st element:		
if count == 0	(pairs)	votes[i+1]
if count > 0 $\wedge$ candidates == votes[i+1]	(pairs)	+1 candidates
if count > 0 $\wedge$ candidates $\neq$ votes[i+1]	+1 pairs	-1 candidates

Practice Questions

1. Trace the max continuous sum algorithm on **[3, -5, 2, 6, -1]**. After each element, write down `cur_sum` (after adding the element, before the reset) and `max_sum`. What does the algorithm return?
2. What does the pseudocode return for **[-3, -1, -2]**? Why does it matter that `max_sum` is updated before `cur_sum` is reset to 0?
3. Trace the Boyer-Moore algorithm on the votes **A B C B B**. After each vote, write down the candidate and count. Which pairs are formed, and which vote is left over?
4. Consider the votes **A A B B**, where A and B each have exactly $n/2$ votes. (a) Run Boyer-Moore on them. What are the returned candidate and the final count? (b) What does that final count tell you about how the votes were paired? (c) Why does the proof of the claim break down here, and why does the problem need “more than $n/2$ ” instead of “at least $n/2$ ”?
5. (Proof) Whenever `cur_sum` goes negative, the algorithm throws away everything before that point and starts fresh. Explain why this can never cause it to miss the best slice. Hint: `cur_sum` is the max sum of a slice ending at i . If it is negative, what do you know about every slice ending at i ? Compare a slice that continues past i with the same slice after removing the part ending at i .

Answer Key

Question 1

i	A[i]	cur_sum (after adding)	max_sum	cur_sum (after reset)
0	3	3	3	3
1	-5	-2	3	0
2	2	2	3	2
3	6	8	8	8
4	-1	7	8	7

The algorithm returns **8**, from the slice [2, 6].

Question 2

It returns **-1**, the largest single element. cur_sum goes -3, -1, -2 (with a reset to 0 after each of the first two), so max_sum ends at -1.

Because max_sum is updated first, negative sums still get recorded. If the reset came first, cur_sum would always be at least 0, and the algorithm would return 0, which is not the sum of any slice in the list.

Question 3

Vote	Candidate	Count
A	A	1
B	A	0
C	C	1
B	C	0
B	B	1

The pairs are (A, B) and (C, B), and the leftover vote is **B**, which is what the algorithm returns.

Question 4

(a) The trace is A (count 1), A (count 2), B (count 1), B (count 0). It returns **A** with a final count of **0**.

(b) A final count of 0 means nothing is left unpaired: every vote was paired with a vote for a different candidate, here (A, B) and (A, B).

(c) The proof says the leftover vote must be for the majority candidate, but there is no leftover vote, so it has nothing to point to. In fact, each pair holds at most one vote for any single candidate, so with $n/2$ pairs no candidate can have more than $n/2$ votes. A has exactly $n/2$, which is not a majority. Requiring “more than $n/2$ ” guarantees that a leftover vote exists, because a candidate with more than $n/2$ votes has too many votes to fit in the pairs.

Question 5

Say `cur_sum` is negative after reading `A[i]`. Since `cur_sum` is the max sum of a slice ending at `i`, every slice ending at `i` has a negative sum.

Now take any slice that starts at or before `i` and continues past `i`. Split it into the part ending at `i` and the part starting at `i+1`. The first part has a negative sum, so removing it makes the total larger. That means the slice starting at `i+1` is better than the original slice.

So no best slice can start at or before `i` and continue past `i`, which means starting fresh at `i+1` loses nothing. Slices that end at or before `i` were already checked by `max_sum`.